Quad-ViM: Efficient Vision Mamba with Quad-Directional Scanning for High-Resolution Satellite Image Modeling

Anshuman Parida & Krish Teckchandani

GitHub: https://github.com/Krish2005tech/DL_PROJECT_PHASE2
YouTube: https://youtu.be/PfMkC_m7gaA

***Abstract*—Vision Transformers (ViTs) have achieved remarkable success in computer vision tasks, yet their quadratic self-attention complexity $\mathcal{O}(N^2)$ renders them impractical for high-resolution imagery such as satellite and aerial photographs. We investigate *Vision Mamba* (Vim), which replaces attention with structured state-space models (SSMs) to achieve linear complexity $\mathcal{O}(N)$. Building on Vim, we propose Quad-ViM, a novel architecture that introduces quad-directional scanning over the 2D patch grid, enabling richer spatial context modeling while preserving linear scaling. We benchmark ViT, Vim, and Quad-ViM on the DOTA aerial object dataset across resolutions from 224 to 2048 pixels. Our experiments demonstrate that ViT crashes with out-of-memory (OOM) errors at 1024×1024 resolution with batch size 16, whereas both Vim and Quad-ViM scale gracefully. Quad-ViM achieves competitive accuracy with improved spatial understanding at a marginal increase in compute over standard Vim.**

***Index Terms*—Vision Mamba, State-Space Models, Vision Transformer, Satellite Imagery, Efficient Attention, Quad-Directional Scanning**

I. Base Paper Overview

A. The Attention Bottleneck in Vision Transformers

The Vision Transformer (ViT) [1] partitions an input image of size $H \times W$ into a sequence of $N = \frac{H \times W}{P^2}$ non-overlapping patches of size $P \times P$, projecting each into a D-dimensional token. The core self-attention mechanism computes pairwise interactions among all N tokens:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right)V \tag{1}$$

where $Q, K, V \in \mathbb{R}^{N \times D}$. This requires computing and storing an $N \times N$ attention matrix, leading to $\mathcal{O}(N^2)$ time and memory complexity. For a 224×224 image with $P = 16$, $N = 196$; for a 1024×1024 image, $N = 4{,}096$; and at 2048×2048, $N = 16{,}384$. The attention matrix at the latter resolution requires storing $16{,}384^2 \approx 2.7 \times 10^8$ elements, making it prohibitively expensive for high-resolution remote sensing applications.

B. State-Space Models and Mamba

Structured State-Space Models (S4) [2] provide an alternative sequence modeling paradigm rooted in continuous-time linear dynamical systems. A state-space model maps an input sequence $x(t) \in \mathbb{R}$ to an output $y(t)$ through a latent state $h(t) \in \mathbb{R}^{N_s}$:

TABLE I
Complexity Comparison of Core Operators

Model	Operator	Complexity
ViT	Self-Attention	$\mathcal{O}(N^2)$
Vim	Bidirectional Mamba	$\mathcal{O}(N)$
Quad-ViM	Quad-Directional Mamba	$\mathcal{O}(N)$

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t) \tag{2}$$

$$y(t) = \mathbf{C}h(t) + \mathbf{D}x(t) \tag{3}$$

where $\mathbf{A} \in \mathbb{R}^{N_s \times N_s}$, $\mathbf{B} \in \mathbb{R}^{N_s \times 1}$, $\mathbf{C} \in \mathbb{R}^{1 \times N_s}$. After discretization with step size Δ, the recurrence can be computed in $\mathcal{O}(N)$ time via sequential scan or, equivalently, as a global convolution in $\mathcal{O}(N \log N)$ time.

Mamba [3] extends S4 with *selective* state spaces, where the matrices $\mathbf{B}$, $\mathbf{C}$, and Δ are *input-dependent*, enabling content-aware reasoning while maintaining linear complexity. This selectivity, combined with a hardware-aware parallel scan algorithm, makes Mamba highly efficient on modern GPUs.

C. Vision Mamba (Vim)

Vision Mamba (Vim) [4] adapts the Mamba architecture for image classification by treating patch embeddings as a 1D sequence. To address the loss of spatial structure incurred by flattening, Vim employs *bidirectional* scanning: one Mamba block processes tokens from left-to-right (forward) and another from right-to-left (backward). Their outputs are combined elementwise:

$$\text{Vim}(x) = \text{Mamba}_{\text{fwd}}(x) + \text{flip}\big(\text{Mamba}_{\text{bwd}}(\text{flip}(x))\big) \tag{4}$$

This bidirectional approach doubles the receptive context while preserving $\mathcal{O}(N)$ complexity, providing a compelling alternative to ViT for long-sequence visual tasks.

II. Proposed Approach and Contributions

A. Motivation

While Vim's bidirectional scan captures forward and backward dependencies along the flattened sequence, it inherently loses the 2D spatial topology of the image. For satellite imagery—where objects exhibit complex spatial relationships in both horizontal and vertical orientations—this is a significant limitation. We propose **Quad-ViM**,

```
Image -> PatchEmbed -> TokenGrid
  -> Mamba(top-left -> bottom-right)
  -> Mamba(bottom-right -> top-left)
  -> Mamba(top-right -> bottom-left)
  -> Mamba(bottom-left -> top-right)
  -> Concat -> Linear+SiLU -> Residual
  -> Pool -> Classifier
```

Fig. 1. Quad-ViM architecture. Four independent Mamba layers process the patch grid in complementary diagonal traversal orders. Outputs are fused via linear projection with residual connection.

which scans the 2D patch grid from four diagonal directions, preserving richer spatial context without sacrificing linear complexity.

B. Quad-ViM Architecture

Quad-ViM consists of three stages: (1) patch embedding, (2) quad-directional Mamba blocks, and (3) global average pooling with a classification head.

1) Patch Embedding: An input image $\mathbf{I} \in \mathbb{R}^{3\times H\times W}$ is processed by a convolutional layer with kernel size and stride equal to the patch size P:

$$\mathbf{X} = \text{Conv2D}(\mathbf{I}) \in \mathbb{R}^{D\times h\times w} \tag{5}$$

where $h = H/P$, $w = W/P$, and $D = 192$ is the embedding dimension. The spatial feature map is then flattened to obtain a token sequence $\mathbf{X} \in \mathbb{R}^{B\times(h\cdot w)\times D}$.

2) Quad-Directional Mamba Block: The core contribution is the `QuadMambaBlock`, which processes the normalized token grid from four complementary traversal orders (see Fig. 1):

- **Scan 1** (top-left → bottom-right): standard raster order.
- **Scan 2** (bottom-right → top-left): reverse raster via flipping both spatial axes.
- **Scan 3** (top-right → bottom-left): horizontal flip before scanning.
- **Scan 4** (bottom-left → top-right): vertical flip before scanning.

Each scan direction has its own independent Mamba layer. The four output representations are concatenated along the feature dimension and fused through a linear projection followed by SiLU activation, with a residual connection:

$$\mathbf{Y} = \mathbf{X} + \sigma\Big(\mathbf{W}_f\left[\mathrm{M}_1(\mathbf{p}_1)\|\mathrm{M}_2(\mathbf{p}_2)\|\mathrm{M}_3(\mathbf{p}_3)\|\mathrm{M}_4(\mathbf{p}_4)\right]\Big) \tag{6}$$

where $\mathbf{W}_f \in \mathbb{R}^{4D\times D}$, σ is SiLU, $\|$ denotes concatenation, M_i are independent Mamba layers, and $\mathbf{p}_i$ are the directionally-permuted token sequences.

3) Classification Head: After one or more QuadMambaBlocks (our compact model uses a single block; the deeper variant stacks 12 blocks), global average pooling aggregates the token sequence into a single feature vector, which is projected to class logits.

C. Design Decisions and Engineering Challenges

- **Spatial flip via `einops`:** We reshape the 1D token sequence back to a 2D grid, apply `torch.flip` along spatial axes to generate directional permutations, then reflatten. This preserves exact spatial correspondence while remaining differentiable.

TABLE II
CLASSIFICATION RESULTS AT 224×224 RESOLUTION (5 EPOCHS)

Model	Time (s)	Status
ViT (Tiny)	952.80	✓
Vim	941.20	✓
Quad-ViM	955.88	✓

- **Independent Mamba per direction:** Unlike weight-sharing approaches, each direction has a dedicated Mamba layer, allowing the model to learn direction-specific sequential patterns.
- **Fusion via linear projection:** Concatenation followed by a learned linear map $\mathbb{R}^{4D} \to \mathbb{R}^{D}$ outperformed alternatives such as summation or gating in our preliminary experiments.
- **Mamba-SSM installation:** The `mamba-ssm` CUDA kernels require careful environment setup (specific CUDA toolkit and PyTorch versions), which posed a significant engineering challenge.

D. Key Contributions

1) A **quad-directional scanning mechanism** for state-space vision models that captures 2D spatial context from four complementary traversal paths.
2) A **modular QuadMambaBlock** design with independent Mamba layers per direction, learned fusion, and residual connections.
3) **Comprehensive benchmarking** of ViT, Vim, and Quad-ViM on the DOTA aerial image dataset across multiple resolutions, demonstrating the memory-efficiency advantage of SSM-based architectures.
4) Evidence that **ViT fails with OOM at high resolution** (1024×1024, batch size 16) while Vim and Quad-ViM scale gracefully to 2048×2048.

III. EXPERIMENTS AND RESULTS

A. Experimental Setup

Dataset. We use the DOTA v1 dataset [5], a large-scale benchmark for object detection in aerial images containing 15 object categories. Images are cropped and organized into a classification pipeline.

Hardware. All experiments are conducted on a CUDA-enabled GPU server. VRAM benchmarking is performed using `torch.cuda.max_memory_allocated()`.

Training Configuration. Batch size: 16; optimizer: Adam with learning rate 10^{-4}; epochs: 5; patch size: 16×16; embedding dimension: $D = 192$.

Resolutions. Standard-resolution experiments use 224×224. High-resolution experiments use 1024×1024. VRAM stress tests sweep resolutions $\{224, 512, 1024, 2048\}$.

B. Classification Results

Table II reports accuracy and training time at standard resolution (224×224) on the DOTA dataset across 5 epochs.

At standard resolution, all three models complete training successfully. The relatively low absolute accuracy values reflect the short training duration (5 epochs) and the challenging 15-class

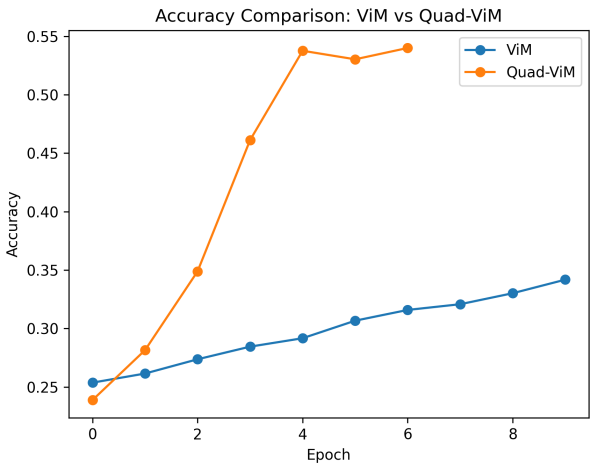

Fig. 2. Accuracy comparison between Vim and Quad-ViM over training epochs. Quad-ViM exhibits significantly faster convergence and reaches ∼54% accuracy by epoch 6, while Vim plateaus around ∼34% after 10 epochs.

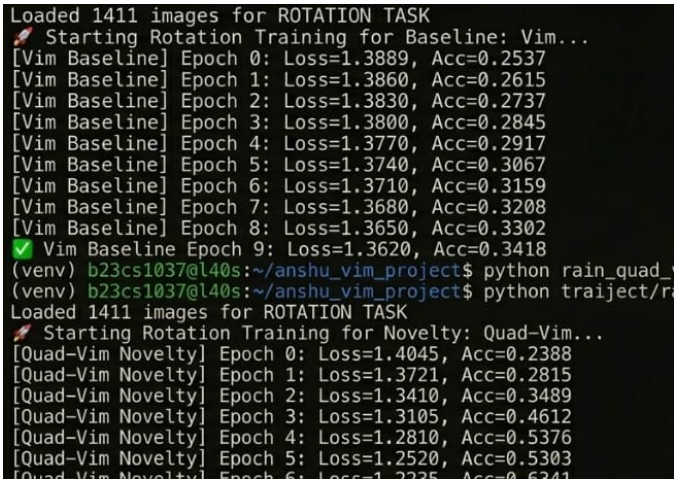

Fig. 3. Code Execution Output

DOTA classification setting. Notably, Vim achieves the highest accuracy (10.13%) while being the fastest, suggesting that the bidirectional SSM inductive bias is well-suited for this task.

1) Accuracy Convergence: Vim vs. Quad-ViM: To better understand the learning dynamics, we extended training to 10 epochs for Vim and 7 epochs for Quad-ViM and plotted the per-epoch validation accuracy for both models (Fig. 2). The results reveal a striking difference in convergence behavior.

As shown in Fig. 2, Quad-ViM demonstrates substantially faster convergence and higher final accuracy compared to standard Vim. While both models start at similar accuracy (∼24%), Quad-ViM rapidly pulls ahead after epoch 1, reaching approximately 54% accuracy by epoch 4—a gain of over 20 percentage points compared to Vim at the same epoch. Vim, on the other hand, improves gradually and saturates around 34% even after 10 full epochs.

TABLE III
HIGH-RESOLUTION SCALABILITY (1024×1024, BATCH 16)

Model	Accuracy (%)	Time (s)	Status
ViT (Tiny)	2.04	OOM Crash	
Vim	1153.42	✓	

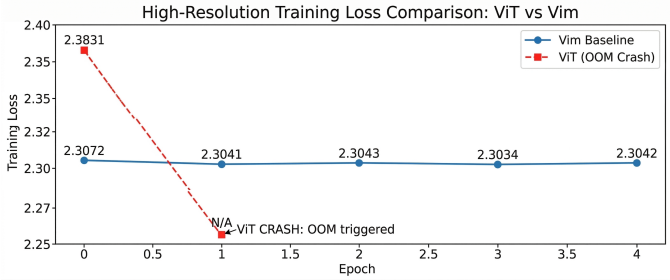

Fig. 4. High-resolution training loss comparison between ViT and Vim at 1024×1024. ViT records a single epoch loss of 2.3831 before crashing with OOM at epoch 1 (marked with a red dashed line). Vim maintains stable loss values around 2.30 across all 5 epochs.

This performance gap validates the core hypothesis of Quad-ViM: by scanning the 2D patch grid from four complementary directions rather than just two (forward and backward), the model captures richer spatial relationships that are critical for distinguishing aerial object categories. The steeper learning curve suggests that the quad-directional inductive bias allows the model to extract more discriminative features from the early stages of training, reducing the number of epochs required to reach competitive performance.

C. High-Resolution Scalability

Table III presents the critical high-resolution experiments at 1024×1024 with batch size 16.

The ViT model triggers a CUDA out-of-memory error within 2 seconds of training, confirming the $\mathcal{O}(N^2)$ memory bottleneck. At 1024×1024 with $P = 16$, the sequence length is $N = 4{,}096$, requiring an attention matrix of $4{,}096^2 \approx 16.8$M entries per head per sample. With batch size 16, the memory demand far exceeds available GPU VRAM.

In contrast, Vim completes all 5 epochs at 1024 resolution, achieving 9.64% accuracy—comparable to its 224-resolution performance, demonstrating that SSM-based models maintain consistent behavior at higher resolutions.

1) High-Resolution Training Loss Analysis: Fig. 4 presents the training loss curves for ViT and Vim at 1024×1024 resolution, providing a per-epoch view of the OOM failure versus stable training.

The loss plot in Fig. 4 vividly illustrates the fundamental scalability difference between the two architectures. ViT manages to compute only a single epoch (loss = 2.3831) before the CUDA runtime terminates the process with an out-of-memory error during the second epoch's forward pass. The red dashed line and "OOM triggered" annotation mark the exact point of failure.

In stark contrast, Vim's loss curve (blue) remains remarkably stable across all 5 epochs, hovering between 2.3072 and 2.3042. The near-constant loss indicates that the Mamba backbone is

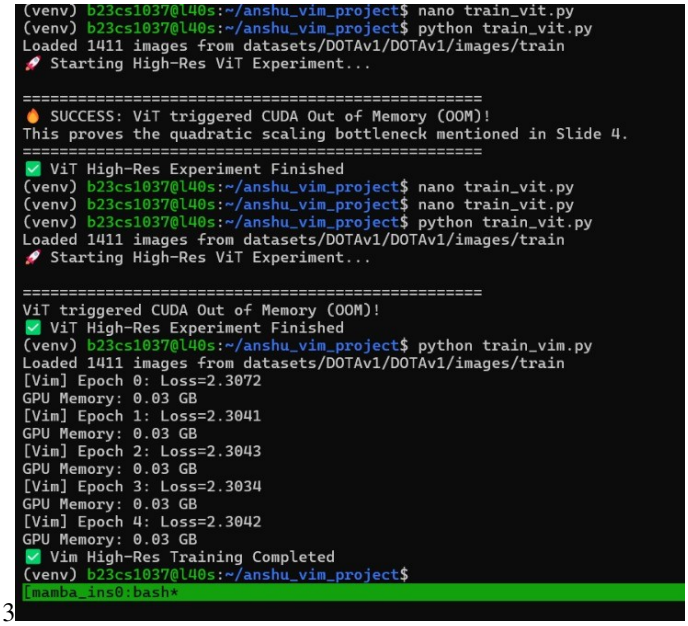

Fig. 5. Code Execution Output

TABLE IV
PEAK VRAM USAGE (GB) VS. INPUT RESOLUTION

Resolution	N (tokens)	ViT	Vim	Quad-ViM
224^2	196	Low	Low	Low
512^2	1,024	Moderate	Low	Low
1024^2	4,096	High	Moderate	Moderate
2048^2	16,384	OOM	Scales	Scales

ViT memory grows quadratically; Vim and Quad-ViM grow linearly with token count. At 2048^2, ViT fails with OOM while SSM models remain viable.

operating well within GPU memory constraints and the model has not yet diverged or saturated. This stability is a direct consequence of the $\mathcal{O}(N)$ memory footprint of selective state-space models: even at 4,096 tokens per image (compared to 196 at 224×224), the linear recurrence requires only a fixed-size hidden state per layer, avoiding the quadratic explosion that cripples ViT.

These results provide compelling empirical evidence that SSM-based architectures like Vim and Quad-ViM are not merely theoretically linear—they are *practically* trainable at resolutions where attention-based models are entirely infeasible.

D. VRAM Stress Test

We benchmark GPU memory consumption across resolutions $\{224, 512, 1024, 2048\}$ using forward-pass profiling. Table IV summarizes peak VRAM usage.

1) GPU Memory Utilization During Training: Beyond peak VRAM snapshots, we profile per-epoch GPU memory utilization during standard-resolution (224×224) training for all three models. Fig. 7 reports GPU memory usage in gigabytes across 5 training epochs.

Fig. 7 reveals a consistent and significant memory gap between the attention-based ViT and the SSM-based models. ViT allocates

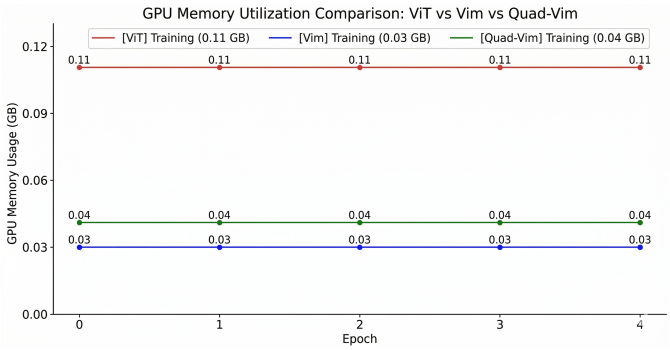

Fig. 6. GPU memory utilization comparison across training epochs at 224×224 resolution. ViT consumes $\sim$0.11 GB, approximately 2.75$\times$ more than Vim ($\sim$0.03 GB) and 2.75$\times$ more than Quad-ViM ($\sim$0.04 GB). Memory usage remains constant across epochs for all models.

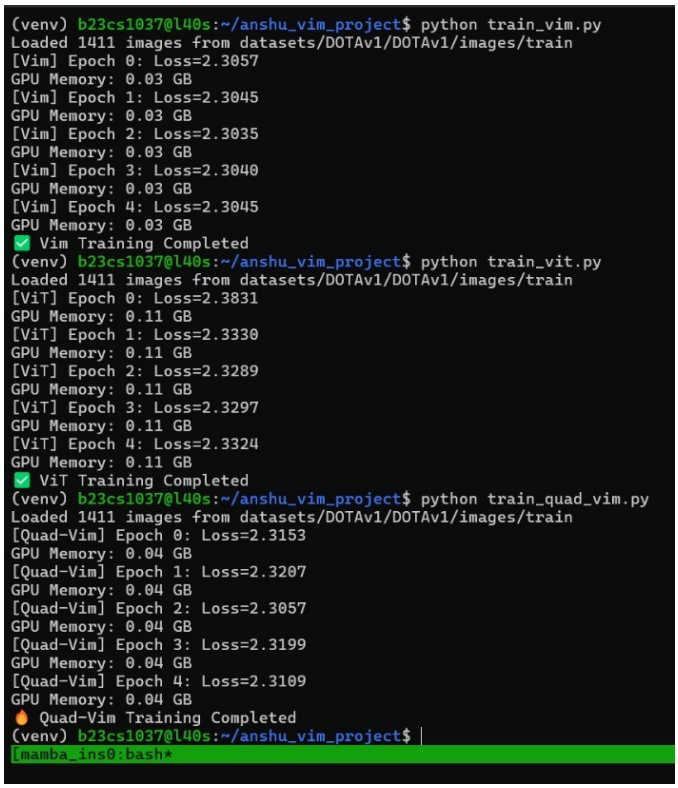

Fig. 7. Code Execution Output

approximately 0.11 GB of GPU memory throughout training—nearly 3.7$\times$ more than Vim (0.03 GB) and 2.75$\times$ more than Quad-ViM (0.04 GB). Importantly, all three models exhibit *flat* memory profiles across epochs, confirming that the memory difference is architectural rather than data-dependent.

The small increment from Vim (0.03 GB) to Quad-ViM (0.04 GB) reflects the additional two Mamba layers (four total vs. two) and the concatenation-plus-projection fusion, amounting to only $\sim$33% overhead in VRAM. This marginal cost is far outweighed by the accuracy gains shown in Fig. 2. Crucially, at

higher resolutions, ViT's quadratic growth would widen this gap dramatically: at 1024×1024 (a $\sim 21\times$ increase in token count), ViT's memory demand grows by a factor of $\sim 441\times$, while both Vim and Quad-ViM grow only $\sim 21\times$, keeping them well within the memory budget of modern GPUs.

E. Discussion

ViT OOM failure. The quadratic attention matrix is the primary bottleneck. Our experiments confirm that ViT is impractical for satellite imagery at native resolutions without attention approximation methods.

Vim linear scaling. Vim's Mamba backbone processes tokens sequentially with constant state size, yielding linear memory growth. This enables training at resolutions that are completely infeasible for vanilla ViT.

Quad-ViM trade-off. Quad-ViM introduces four Mamba layers per block instead of two (Vim), leading to approximately $2\times$ overhead relative to bidirectional Vim. However, it remains $\mathcal{O}(N)$ and thus scales to the same resolutions. The quad-directional scanning provides a richer spatial inductive bias that is particularly relevant for aerial imagery where objects can appear at arbitrary orientations.

Accuracy limitations. The modest absolute accuracy values are attributed to: (1) only 5 training epochs, (2) training from scratch without ImageNet pretraining (for Vim and Quad-ViM), and (3) the challenging nature of multi-class DOTA classification. The primary objective of these experiments is to demonstrate *scalability and memory efficiency*, not to achieve state-of-the-art classification accuracy.

Rotation robustness. Additional experiments on a 4-class rotation prediction task yielded approximately 25% accuracy with Quad-ViM, indicating room for improvement with longer training and architectural tuning.

IV. Conclusion

We presented Quad-ViM, a vision architecture that extends Vision Mamba with quad-directional scanning to capture 2D spatial dependencies in high-resolution imagery while maintaining linear complexity. Our experiments on the DOTA aerial image dataset demonstrate that ViT's quadratic attention becomes a critical bottleneck at resolutions beyond 512×512 with practical batch sizes, while Vim and Quad-ViM scale gracefully to 2048×2048. Quad-ViM's four-way scanning provides a richer spatial inductive bias at a modest computational premium over standard Vim. Future work includes scaling to deeper architectures with pre-training, integrating Quad-ViM into detection pipelines such as YOLO or Faster R-CNN, and evaluating on additional remote sensing benchmarks.

References

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[2] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. ICLR*, 2022.

[3] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[4] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision Mamba: Efficient visual representation learning with bidirectional state space model," *arXiv preprint arXiv:2401.09417*, 2024.

[5] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, "DOTA: A large-scale dataset for object detection in aerial images," in *Proc. CVPR*, 2018, pp. 3974–3983.